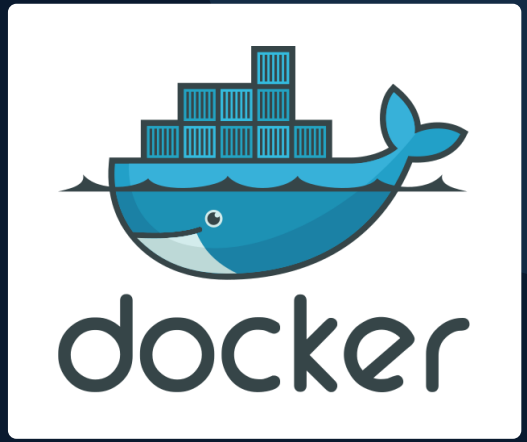
3IW — ESGI · CONTENEURISATION · 14 HEURES

Docker

De zéro à Docker Compose

Comprendre les conteneurs, écrire un Dockerfile propre et orchestrer une stack complète avec Compose.



Aloïs Marcellin — Lead developer fullstack, Wizards Technologies
4 séances de 3h30 · 40 % d'apports, 60 % de pratique

SOMMAIRE

Au programme

1

Les fondamentaux

Images, conteneurs, volumes, ports. Toute la CLI dont vous aurez besoin au quotidien.

2

Dockerfile

Construire vos propres images, proprement :
FROM, RUN, CMD, ENTRYPOINT, COPY, ARG…

3

Docker Compose

Orchestrer plusieurs conteneurs en un seul fichier
et lancer une stack entière d'une commande.

Chaque partie se termine par un TP à faire sur votre machine. Le fil rouge : une petite application web que l'on conteneurise du début à la fin.

PRÉSENTATION

Qui suis-je ?

Aloïs Marcellin

Lead developer fullstack — Wizards Technologies

- Conception et mise en production d'applications web
- Docker au quotidien : dev local, CI/CD, déploiement
- Intervenant en école d'ingénieurs

CE QUE J'ATTENDS DE VOUS

- Coupez-moi la parole
- Testez tout en direct
- Cassez vos conteneurs, c'est fait pour

AVANT DE COMMENCER

Test de positionnement

Cinq minutes, sans enjeu de note : il me sert à caler le rythme du cours sur votre niveau réel.

1. **Vous n'avez jamais lancé de conteneur**
   Parfait, c'est exactement le public visé.

2. **Vous avez déjà fait un docker run**
   On ira vite sur la partie 1, on creusera le Dockerfile.

3. **Vous écrivez déjà des Compose**
   Venez me voir, je vous donnerai des exercices bonus.

1

Les fondamentaux

Ce qu'est réellement un conteneur, en quoi il diffère d'une VM, et les commandes que vous taperez tous les jours.

DÉFINITION

Docker, qu'est-ce que c'est ?

Un outil qui permet d'empaqueter une application avec tout ce dont elle a besoin, puis de la lancer à l'identique sur n'importe quelle machine.

R Reproductible

La même image tourne sur votre portable, chez votre collègue et en production. Fini les écarts d'environnement.

L Léger

Pas d'OS invité à démarrer : un conteneur se lance en quelques centaines de millisecondes.

I Isolé

Chaque conteneur a ses processus, son réseau et son système de fichiers. Une appli qui casse ne casse pas les autres.

MOTIVATION

Le problème que Docker résout

SANS DOCKER

« Chez moi ça marche »

- Une page d'installation à suivre pour chaque poste
- Deux versions de Node, trois de Python, un PHP global
- Un serveur de prod configuré à la main, jamais reproductible
- Une journée perdue à chaque arrivée dans l'équipe

AVEC DOCKER

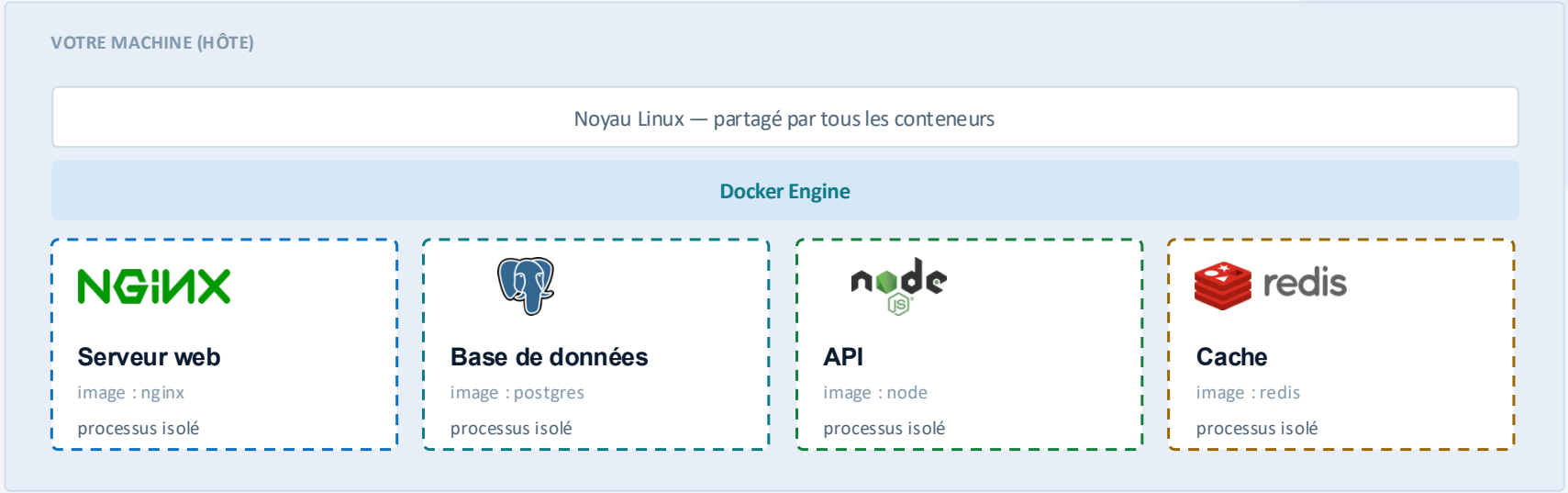
« git clone && docker compose up »

- L'environnement est décrit dans un fichier versionné
- Chaque service a ses propres dépendances, sans conflit
- La même définition sert en local, en CI et en production
- Un nouvel arrivant est opérationnel en une commande

CONCEPT

Vous avez dit conteneur ?

Un conteneur est un processus isolé du reste du système sur lequel il tourne. Pas une machine : un processus.

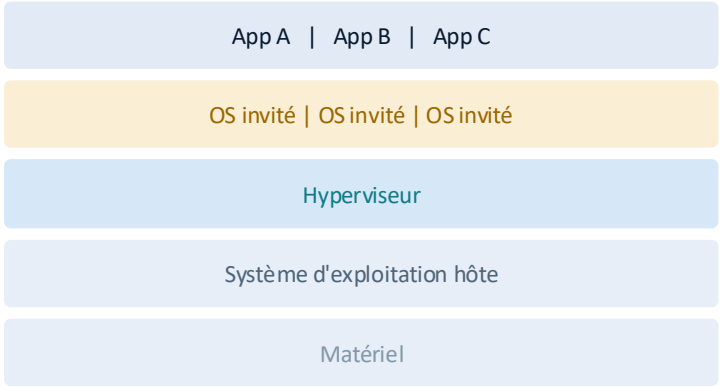


Les quatre conteneurs partagent le même noyau : aucun n'embarque son propre système d'exploitation.

COMPARAISON

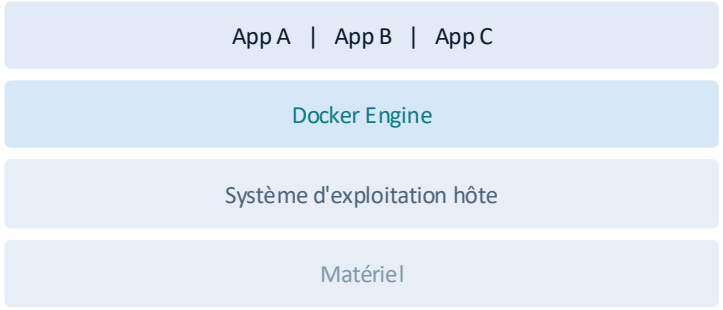
Conteneur ou machine virtuelle ?

MACHINE VIRTUELLE



Chaque VM embarque un OS complet : plusieurs Go, démarrage en dizaines de secondes.

CONTENEURS



Les conteneurs partagent le noyau de l'hôte : quelques Mo, démarrage quasi instantané.

Les deux approches sont complémentaires : en production, vos conteneurs tournent presque toujours dans des VM.

LE DUO À NE PAS CONFONDRE

Image et conteneur

IM L'image

Un modèle figé, en lecture seule. C'est la recette : le système de fichiers et la commande à lancer.

CT Le conteneur

Une instance en cours d'exécution de cette image, avec sa propre couche modifiable par-dessus.

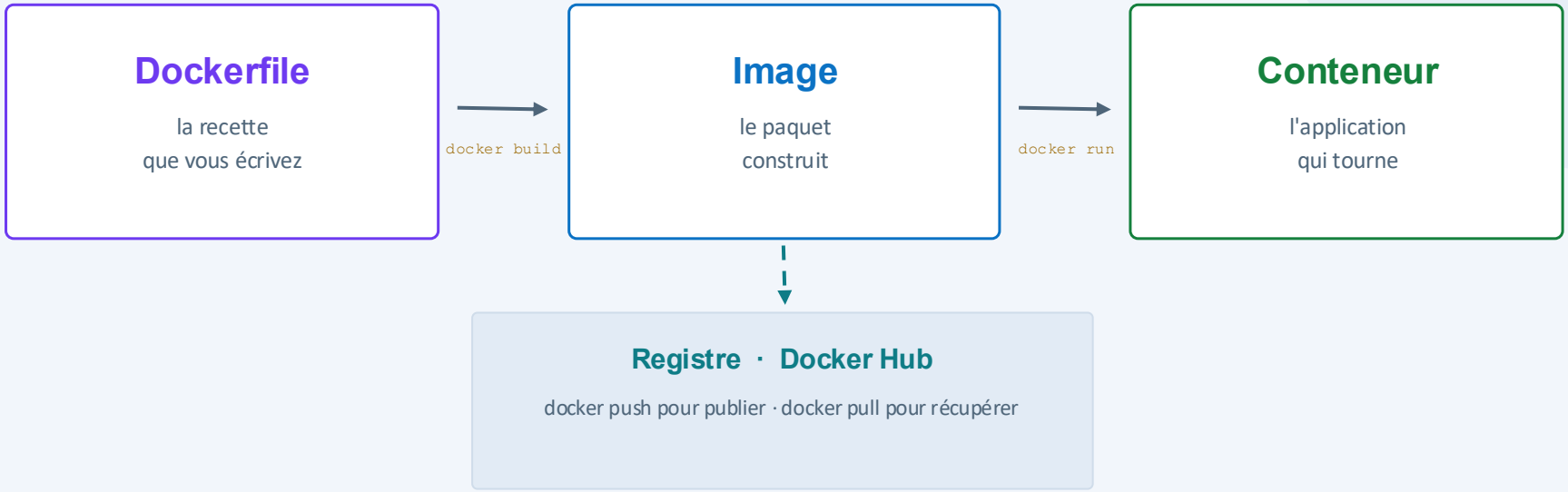


Une image, autant de conteneurs que vous voulez. L'image, elle, n'est jamais modifiée.

VUE D'ENSEMBLE

Le cycle de vie

Tout le cours tient dans ce schéma. Nous allons le parcourir de gauche à droite.



MISE EN ROUTE

L'installation

OÙ TÉLÉCHARGER

docs.docker.com/engine/install

Docker Desktop sur Windows et macOS (il embarque une VM Linux). Docker Engine directement sur une distribution Linux.

VÉRIFIER QUE TOUT MARCHE

```
docker run hello-world
```

Si vous voyez « Hello from Docker! », l'installation est bonne et le démon répond.

LES DEUX ERREURS DE DÉBUT

- « Cannot connect to the Docker daemon » : le service n'est pas démarré. Lancez Docker Desktop, ou sudo systemctl start docker.
- « permission denied » sous Linux : ajoutez votre utilisateur au groupe docker, puis rouvrez votre session.

Trouver de l'aide

docker --help

La liste complète des commandes. Fonctionne aussi sur chaque sous-commande.

```
docker --help
docker run --help
docker compose --help
```

Documentation complète : docs.docker.com/reference/cli/docker

docker info

L'état de votre installation, en une commande.

- Conteneurs lancés, en pause, arrêtés
- Nombre d'images stockées localement
- Version du moteur et du système
- Mémoire et CPU disponibles
- Utilisateur connecté au registre

Astuce : docker version donne les versions du client et du démon séparément. Si le démon ne répond pas, c'est ici que vous le verrez en premier.

LA COMMANDE CENTRALE

docker run

Crée un conteneur à partir d'une image, puis le démarre. Toutes les options se combinent.

```
docker run [options] image[:tag] [commande] [arg...]
```

`-it`	Mode interactif avec un terminal. Indispensable pour obtenir un shell dans le conteneur.
`-d`	Détaché : le conteneur tourne en arrière-plan et vous rend la main.
`--name`	Donne un nom lisible au conteneur, au lieu d'un nom généré aléatoirement.
`-p`	Publie un port du conteneur sur l'hôte.
`-v`	Monte un dossier de l'hôte dans le conteneur.
`--rm`	Supprime automatiquement le conteneur dès qu'il s'arrête. Parfait pour les tests.

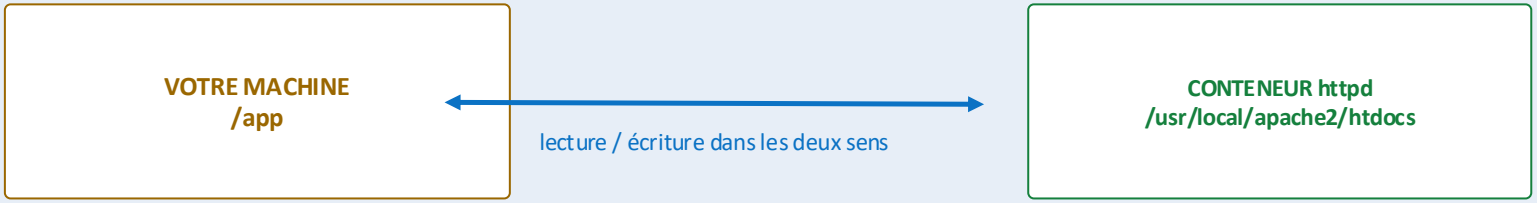
```
docker run -d --name mon-site -p 8080:80 nginx:1.27
```

Les volumes

Par défaut, tout ce qu'écrit un conteneur disparaît avec lui. Un volume relie un dossier de l'hôte à un dossier du conteneur.

```
docker run -v /app:/usr/local/apache2/htdocs httpd
```

dossier de l'hôte — dossier dans le conteneur



Vous modifiez un fichier dans votre éditeur : le conteneur voit le changement immédiatement.

Deux usages : le bind mount (un chemin de votre disque, idéal en développement) et le volume nommé (géré par Docker, idéal pour les données d'une base).

Les ports

Un conteneur est isolé sur le réseau. Sans publication de port, rien de ce qu'il expose n'est joignable depuis votre navigateur.

```
docker run -p 8080:80 httpd:latest
```

port sur votre machine — port dans le conteneur



On lit toujours -p hôte:conteneur. Les deux numéros n'ont pas besoin d'être identiques : rien n'empêche de publier le port 80 du conteneur sur le port 3000 de votre machine.

Observer ce qui tourne

docker ps

Liste les conteneurs en cours d'exécution : identifiant, image, ports publiés, nom.

```
docker ps
docker ps -a   # y compris les arrêtés
```

docker logs

Affiche la sortie du conteneur. C'est le premier endroit à regarder quand quelque chose ne marche pas.

```
docker logs mon-site
docker logs -f --tail 50 mon-site
```

docker exec

Lance une commande dans un conteneur déjà démarré, sans l'interrompre.

```
docker exec -it mon-site /bin/sh
```

docker exec ne relance pas la commande si le conteneur redémarre : ce n'est pas un moyen de configurer une image.

START · STOP · RM

Piloter les conteneurs

docker start

Redémarre un ou plusieurs conteneurs arrêtés. Le conteneur retrouve son état de fichiers.

docker stop

Arrête proprement un ou plusieurs conteneurs en cours d'exécution.

docker rm

Supprime un conteneur, par son nom ou son identifiant. Un conteneur en cours ne peut pas être supprimé : ajoutez -f pour forcer.

FAIRE LE MÉNAGE

```
docker stop $(docker ps -q)      # arrête tout ce qui tourne
docker container prune           # supprime tous les conteneurs arrêtés
```

Gérer les images

docker images

- Affichage de la plus récente à la plus ancienne
- Dépôt, tag et taille de chaque image
- La taille affichée inclut les images parentes, souvent partagées entre plusieurs images
- Une même image apparaît autant de fois qu'elle a de tags
- -a montre aussi les couches intermédiaires mises en cache

```
docker images
docker image prune
```

docker pull

- Exécuté automatiquement par docker run si l'image est absente en local
- Sans tag précisé, c'est latest qui est récupéré

docker rmi

- Refuse de supprimer une image utilisée par un conteneur existant
- -f force la suppression : les conteneurs deviennent orphelins mais continuent de tourner

Le Docker Hub

Un registre public d'images. Un compte gratuit sur hub.docker.com suffit ; l'authentification est obligatoire pour publier.

`docker login`

Authentification auprès du registre. Nécessaire pour pousser, et pour tirer une image privée.

`docker search`

Recherche une image sur le Hub. L'option --filter permet de ne garder que les images officielles ou les mieux notées.

`docker commit`

Crée une image à partir d'un conteneur que vous avez modifié à la main. Pratique pour dépanner, à proscrire en production.

`docker push`

Publie une image vers le Hub. Le nom doit respecter la forme utilisateur/image:tag.

```
docker tag alpine-dm votre-login/alpine-dm:1.0   &&   docker push votre-login/alpine-dm:1.0
```

AIDE-MÉMOIRE

Récapitulatif de la partie 1

Découvrir

`docker info`
état de l'installation

`docker search nginx`
chercher une image

`docker images`
images en local

`docker ps -a`
tous les conteneurs

Lancer

`docker pull nginx`
récupérer une image

`docker run -d -p 8080:80 nginx`
démarrer en arrière-plan

`docker run -it alpine /bin/sh`
obtenir un shell

`docker exec -it web /bin/sh`
entrer dans un conteneur

Nettoyer

`docker stop web`
arrêter

`docker rm web`
supprimer un conteneur

`docker rmi nginx`
supprimer une image

`docker system prune`
tout nettoyer

TRAVAUX PRATIQUES

TP 1 — Prise en main

Objectif : manipuler un conteneur de bout en bout, de la récupération de l'image jusqu'à sa publication sur le Hub.

1 Terminal 1

- Récupérer l'image alpine
- Vérifier qu'elle est bien présente en local
- Démarrer un conteneur interactif avec un shell
- Lister les processus en cours : vous devez en voir deux

2 Terminal 2

- Lister les conteneurs lancés et relever le nom du vôtre
- Y ajouter un processus : un ping vers www.google.com
- Revenir au terminal 1 : vous devez maintenant voir trois processus
- Toujours dans le conteneur, créer un fichier à votre nom

Énoncé complet : exercices/Partie 1/Exercice_1.pdf. Toutes les commandes utiles sont sur la slide « Récapitulatif de la partie 1 » ; si vous bloquez, la réponse est presque toujours dans docker ps ou docker logs.

TRAVAUX PRATIQUES

TP 1 — Publier son image

3

Créer votre image

Depuis le terminal 2, créez une image à partir du conteneur que vous venez de modifier. Nommez-la votre-login/alpine-dm.

4

Publier sur le Hub

Connectez-vous au Docker Hub, poussez votre image, puis vérifiez sur hub.docker.com qu'elle apparaît bien.

5

Tout supprimer

Arrêtez et supprimez votre conteneur, puis retirez de votre machine l'image alpine et votre image alpine-dm.

6

Repartir de zéro

Démarrez un nouveau conteneur depuis votre-login/alpine-dm : Docker doit aller la rechercher sur le Hub. Vérifiez que votre fichier est toujours là.

L'étape 6 est celle qui compte : elle prouve que votre image vit désormais indépendamment de votre machine.

TRAVAUX PRATIQUES

TP 2 — Cycle de vie d'un conteneur

Objectif : piloter un serveur web du démarrage à la suppression — port publié, dossier monté, logs, nettoyage.

1	**Créer un conteneur**	Un conteneur nginx nommé mon-site-web. Ouvrez http://localhost : que se passe-t-il, et pourquoi ?
2	**Publier un port**	Rendez le serveur accessible sur le port 8080 de votre machine, et vérifiez la colonne PORTS de docker ps.
3	**Monter un volume**	Montez un dossier de votre machine sur /usr/share/nginx/html, puis modifiez la page dans votre éditeur.
4	**Entrer dans le conteneur**	Ouvrez un shell à l'intérieur, vérifiez le contenu du dossier monté, ressortez sans arrêter le conteneur.
5	**Observer et nettoyer**	Lisez les logs, arrêtez le conteneur, supprimez-le, et vérifiez qu'il ne reste rien.

Énoncé complet : exercices/Partie 1/Exercice_2.pdf — corrigé : Correction_2.pdf

2

Le Dockerfile

Arrêter de bricoler à la main : décrire son image dans un fichier versionné, lisible et reconstructible à l'identique.

DÉFINITION

Le Dockerfile

Un fichier texte qui liste, ligne par ligne, les instructions permettant à Docker de construire une image automatiquement.

```
FROM node:22-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY . .
EXPOSE 3000
CMD ["node", "server.js"]
```

POURQUOI C'EST MIEUX QUE DOCKER COMMIT

- Le fichier vit dans votre dépôt Git : l'image est versionnée en même temps que le code
- N'importe qui peut reconstruire exactement la même image, y compris votre serveur d'intégration continue
- Chaque instruction est explicite : on relit la recette au lieu de deviner ce qui a été tapé dans un shell

CONSTRUIRE L'IMAGE

docker build

```
docker build -t <utilisateur>/<nom-image>:<tag> <chemin>
```

`-t`	Nomme et tague l'image. Sans tag explicite, Docker utilise latest.
`<chemin>`	Le contexte de build : le dossier envoyé au démon. Un simple point désigne le dossier courant.
`-f`	Indique un Dockerfile qui ne s'appelle pas Dockerfile ou qui est ailleurs.
`--build-arg`	Fournit la valeur d'un ARG déclaré dans le Dockerfile.

```
docker build -t monlogin/mon-api:1.0 .
```

LE CONTEXTE DE BUILD

Tout le dossier est envoyé au démon avant la construction. Un dossier node_modules ou .git oublié, et le build rampe.

LA SOLUTION : .dockerignore

```
node_modules
.git
*.log
```

À GARDER EN TÊTE DÈS LA PREMIÈRE LIGNE

Les bonnes pratiques

1 Un conteneur = un processus

Un serveur web, une base de données et un cache font trois conteneurs, pas un seul. C'est Compose qui les réunira.

2 N'installer que le nécessaire

Chaque paquet ajoute du poids, du temps de build et de la surface d'attaque.

3 Partir d'une image officielle et légère

alpine ou une variante -slim plutôt qu'une distribution complète.

4 Figer les versions

node:22-alpine et non node:latest : sinon votre build d'aujourd'hui ne ressemblera pas à celui de la semaine prochaine.

5 Ne jamais mettre de secret dans une image

Mots de passe et clés d'API passent par des variables d'environnement au lancement, pas par le Dockerfile.

6 Ordonner du plus stable au plus changeant

Les instructions du haut doivent bouger le moins souvent possible, pour préserver le cache.

Contre-exemple classique : installer Apache, PHP, MySQL et MinIO dans une seule image. Cela fonctionne — et devient ingérable dès la première mise à jour.

FROM et LABEL

FROM

Toujours la première instruction. Toute image dérive d'une autre image, ou d'une image de base.

```
FROM alpine:3.20
FROM node:22-alpine
FROM python:3.12-slim
```

- Précisez toujours un tag de version
- scratch est l'image vide : rien du tout, à réserver aux binaires autonomes

LABEL

Ajoute des métadonnées à l'image : responsable, version, description, dépôt source.

```
LABEL maintainer="alois@exemple.fr"
LABEL version="1.0"
LABEL description="API du cours"
```

L'ancienne instruction MAINTAINER est dépréciée depuis longtemps : utilisez LABEL maintainer="...".

LE TRIO À NE PAS CONFONDRE

RUN, CMD et ENTRYPOINT

RUN

PENDANT LA CONSTRUCTION

Exécute une commande au moment du build et fige le résultat dans une nouvelle couche de l'image.

```
RUN apk add --no-cache curl
```

CMD

AU DÉMARRAGE, PAR DÉFAUT

Définit la commande lancée au démarrage du conteneur. Elle est remplacée si l'on passe une commande à docker run.

```
CMD ["node", "server.js"]
```

ENTRYPOINT

AU DÉMARRAGE, TOUJOURS

Même rôle, mais ne se remplace pas aussi facilement : le conteneur se comporte comme un exécutable.

```
ENTRYPOINT ["/entrypoint.sh"]
```

LES DEUX ENSEMBLE

ENTRYPOINT ["ping"] + CMD ["localhost"] : le conteneur pingue localhost par défaut, et docker run mon-image google.fr change seulement la cible. ENTRYPOINT fixe le programme, CMD fournit les arguments par défaut.

Forme exec ou forme shell

Ces trois instructions acceptent deux écritures. Le choix n'est pas cosmétique : il change le processus lancé.

FORME EXEC — RECOMMANDÉE

```
CMD ["executable", "param1"]
RUN ["/bin/bash", "/start.sh"]
```

- Aucun shell n'est nécessaire dans l'image
- Le processus devient le PID 1 : il reçoit correctement les signaux d'arrêt
- Pas de surprise d'interprétation

FORME SHELL

```
RUN apk add --no-cache curl
CMD node server.js
```

- Exécutée via /bin/sh -c : pratique pour enchaîner avec && ou utiliser des variables
- Le vrai processus est un enfant du shell et peut ignorer un signal d'arrêt

En pratique : forme shell pour les RUN d'installation, forme exec pour CMD et ENTRYPOINT. C'est ce que font toutes les images officielles.

POURQUOI L'ORDRE COMPTE

Le cache des couches

Chaque instruction crée une couche. Docker réutilise le cache tant que l'instruction et son contexte n'ont pas changé.

```
[+] Building 0.4s
 => [1/4] FROM alpine:3.20                              CACHED
 => [2/4] RUN apk update && apk add ca-certificates bash    CACHED
 => [3/4] COPY package.json ./                          CACHED
 => [4/4] RUN npm ci                                    1.2s
```

Dès qu'une couche change, toutes les suivantes sont reconstruites. D'où la règle : du plus stable au plus changeant.

À ÉVITER

```
COPY . .
RUN npm ci
```

Le moindre changement dans le code invalide le COPY, donc réinstalle toutes les dépendances.

À FAIRE

```
COPY package*.json ./
RUN npm ci
COPY . .
```

Les dépendances ne sont réinstallées que si package.json change.

Regrouper et trier les arguments

Un seul RUN plutôt que dix : une seule couche, et un nettoyage du cache de paquets effectif dans la même instruction.

```
RUN apk add --no-cache \
        bash \
        curl \
        nginx \
        php83 \
        php83-fpm \
        php83-mbstring \
        php83-opcache \
        php83-pdo_mysql \
        supervisor \
 && chown -R nginx:www-data /var/lib/nginx \
 && ln -sf /dev/stdout /var/log/nginx/access.log \
 && ln -sf /dev/stderr /var/log/nginx/error.log
```

- Un paquet par ligne, triés par ordre alphabétique : les ajouts se relisent en un coup d'œil dans une revue de code
- Rediriger les logs vers la sortie standard permet de les lire avec docker logs

COPY et WORKDIR

COPY

Copie des fichiers ou des dossiers du contexte de build vers l'image.

```
COPY package*.json ./
COPY ./src /app/src
COPY --chown=node:node . /app
```

- La source doit être dans le contexte de build : pas de ../ possible
- Les jokers sont autorisés dans les noms de source
- La destination se termine par / si c'est un dossier ou si plusieurs fichiers y sont copiés

WORKDIR

Définit le dossier courant pour toutes les instructions suivantes. Le crée s'il n'existe pas.

```
WORKDIR /app
```

USER

Change l'utilisateur qui exécute la suite. Par défaut un conteneur tourne en root : à éviter.

```
USER node
```

DOCUMENTER LES PORTS

EXPOSE

Indique à Docker — et surtout aux humains — sur quels ports le conteneur écoute.

```
EXPOSE 80
EXPOSE 3000
```

C'est une déclaration d'intention : elle ne publie rien par elle-même.

ATTENTION

EXPOSE ne rend rien accessible depuis votre machine. Il faut toujours publier au lancement.

```
docker run -p 8080:80 img
docker run -P img
```

-P publie tous les ports déclarés sur des ports libres au hasard.

Le port publié sur l'hôte n'a aucune raison d'être le même que celui du conteneur : un service qui écoute sur le port 22 peut très bien être joignable sur le port 34500 de votre machine.

ARG et ENV

ARG

Variable disponible uniquement pendant la construction. Elle n'existe plus dans le conteneur qui tourne.

ENV

Variable d'environnement présente dans l'image et dans tous les conteneurs qui en découlent. Surchargeable avec -e au lancement.

```
FROM ubuntu:24.04
ARG GREET=Bonjour
ENV APP_ENV=production
RUN echo "$GREET" > /greeting.txt
CMD cat /greeting.txt
```

```
docker build -t greets \
  --build-arg GREET=Bonsoir .

docker run greets
# affiche : Bonsoir
```

Ni l'un ni l'autre ne convient pour un secret : la valeur reste lisible dans l'historique de l'image.

TOUT CE QU'ON VIENT DE VOIR

Un Dockerfile complet

```dockerfile
# Image de base légère, version figée
FROM node:22-alpine

LABEL maintainer="alois@exemple.fr"

# Dossier de travail dans le conteneur
WORKDIR /app

# Dépendances d'abord : le cache est préservé
COPY package*.json ./
RUN npm ci --omit=dev

# Puis le code applicatif
COPY . .

ENV NODE_ENV=production
EXPOSE 3000

# On ne tourne pas en root
USER node

CMD ["node", "server.js"]
```

CE QU'IL FAUT REMARQUER

- Le tag de version est explicite
- package.json est copié avant le reste du code
- Le conteneur ne tourne pas en root
- CMD est en forme exec

CONSTRUIRE ET LANCER

```bash
docker build -t mon-api:1.0 .
docker run -d -p 3000:3000 \
  --name api mon-api:1.0
```

AIDE-MÉMOIRE

Récapitulatif de la partie 2

FROM	image de base — toujours en premier
WORKDIR	dossier de travail dans l'image
RUN	commande exécutée pendant le build
ARG	variable disponible au build seulement
USER	utilisateur d'exécution
ENTRYPOINT	programme fixe du conteneur
LABEL	métadonnées (remplace MAINTAINER)
COPY	fichiers du contexte vers l'image
ENV	variable d'environnement persistante
EXPOSE	port d'écoute déclaré (ne publie pas)
CMD	commande par défaut, remplaçable

```
docker build -t nom:tag .   puis   docker run …
```

TRAVAUX PRATIQUES

TP 3 — Votre première image

Objectif : construire une image d'un petit serveur web à partir d'un Dockerfile, et comprendre l'effet du cache.

1	**Le squelette**	Créez un dossier avec un fichier index.html de votre choix et un Dockerfile partant de nginx:alpine.
2	**Construire**	Copiez votre page dans /usr/share/nginx/html, exposez le port 80, puis construisez l'image sous le nom tp2-site:1.0.
3	**Lancer et vérifier**	Démarrez le conteneur en publiant le port 8080 et vérifiez le résultat dans votre navigateur.
4	**Observer le cache**	Modifiez index.html, reconstruisez et lisez la sortie : quelles étapes sont marquées CACHED ? Inversez ensuite l'ordre des instructions et recommencez.
5	**Aller plus loin**	Ajoutez un LABEL maintainer, un ARG pour le titre de la page, et reconstruisez avec --build-arg.

Énoncé complet : exercices/Partie 2/Exercice_3.pdf — corrigé : Correction_3.pdf

TRAVAUX PRATIQUES

TP 4 — Les instructions du Dockerfile

Sept exercices courts et indépendants : une instruction à la fois, vérifiée dans l'image produite.

1	**cowsay**	Une image qui fait parler la vache, avec un message remplaçable au lancement.
2	**EXPOSE**	Déclarer les ports 50 et 70 depuis ubuntu, puis vérifier ce que l'image contient vraiment.
3	**WORKDIR**	Deux répertoires de travail successifs, un fichier dans chacun : où atterrissent-ils ?
4	**USER**	Créer un utilisateur, en faire le compte par défaut, vérifier sous quelle identité on tourne.
5	**Détente**	Un générateur de blagues, à partir d'un dépôt GitHub qu'il faudra ouvrir pour comprendre.
6	**PHP et Apache**	Votre index.html dans l'image officielle PHP, et la page dans le navigateur.
7	**Apache à la main**	Depuis debian : installer, lancer… et trouver pourquoi le conteneur s'arrête aussitôt.

Énoncé complet : exercices/Partie 2/Exercice_4.pdf — corrigé : Correction_4.pdf

TRAVAUX PRATIQUES

TP 5 — Quatre petites applications

Passer de l'image de démonstration à l'image qui fait quelque chose : du code, une API, une page servie.

1	**Du JavaScript dans une image**	Embarquer un index.js et le faire tourner. Puis comparer le poids avec une image officielle de Node.
2	**SPAM**	Un processus qui écrit toutes les secondes : le conteneur vit tant que lui vit.
3	**WHO**	Afficher un utilisateur tiré au hasard d'une API, lisiblement — pas le JSON brut.
4	**Une ville, une page web**	Interroger l'API Géo du gouvernement, la ville étant passée par variable d'environnement, et servir le résultat avec Apache.

Énoncé complet : exercices/Partie 2/Exercice_5.pdf — corrigé : Correction_5.pdf

3

Docker Compose

Décrire toute une stack dans un seul fichier,
et la démarrer d'une commande.

DÉFINITION

Docker Compose, c'est quoi ?

Un outil qui décrit plusieurs conteneurs, leurs réseaux et leurs volumes dans un seul fichier YAML, puis les démarre ensemble.

SANS COMPOSE

```
docker network create app-net
docker run -d --network app-net \
  -e POSTGRES_PASSWORD=... postgres:16
docker run -d --network app-net \
  -p 3000:3000 mon-api:1.0
docker run -d --network app-net \
  -p 8080:80 nginx:alpine
```

AVEC COMPOSE

```
docker compose up -d
```

Le réseau, les variables, les ports et l'ordre de démarrage sont décrits une fois pour toutes dans le fichier, versionné avec le code.

C'est l'outil de référence pour monter un environnement de développement complet, et pour déployer des stacks simples.

COMPOSE.YAML

Anatomie du fichier

```yaml
services:
  web:
    build: .
    ports:
      - "5000:5000"
    depends_on:
      - redis

  redis:
    image: "redis:alpine"

volumes:
  db-data:
```

services

La liste des conteneurs à lancer. Chaque clé est le nom du service — et son nom de machine sur le réseau.

image / build

D'où vient l'image : le Hub, ou un Dockerfile à construire.

volumes · networks

Déclarés à la racine, puis référencés par les services qui les utilisent.

Le fichier s'appelle compose.yaml (docker-compose.yml reste accepté). L'ancienne clé version: n'a plus aucun effet, ne l'écrivez plus. En YAML, l'indentation se fait à l'espace, jamais à la tabulation.

Les commandes

`docker compose up`

Construit si nécessaire, crée le réseau et démarre tous les services.

`docker compose up -d`

La même chose en arrière-plan : la main vous est rendue.

`docker compose down`

Arrête et supprime les conteneurs et le réseau. Ajoutez -v pour supprimer aussi les volumes.

`docker compose ps`

L'état de chaque service de la stack.

`docker compose logs -f api`

Suit les logs d'un service en particulier.

`docker compose exec api sh`

Ouvre un shell dans le conteneur d'un service.

`docker compose build`

Reconstruit les images sans démarrer les services.

`docker compose restart api`

Redémarre un seul service.

Depuis la version 2, Compose est un sous-ensemble de la CLI Docker : on écrit docker compose, sans tiret.

IMAGE OU BUILD

D'où vient l'image ?

DEPUIS UN REGISTRE

`image:`

```
services:
  db:
    image: postgres:16-alpine
```

Pour tout ce que vous ne développez pas vous-même : bases de données, cache, reverse proxy, outillage.

DEPUIS UN DOCKERFILE

`build:`

```
services:
  api:
    build:
      context: ./api
      dockerfile: Dockerfile
```

Pour votre propre code : c'est le Dockerfile de la partie 2 qui est construit.

Une stack réelle mélange les deux : votre API se construit, la base de données et le cache viennent du Hub. Une forme courte existe aussi : `build: .` lorsque le Dockerfile est dans le dossier courant.

Ports et variables d'environnement

ports

L'équivalent de -p. Toujours hôte:conteneur.

```yaml
ports:
  - "8000:8000"
  - "3000-3005"
  - "127.0.0.1:8001:8001"
  - "9090-9091:8080-8081"
  - "6060:6060/udp"
```

- Les guillemets évitent que YAML interprète 22:22 comme un sexagésimal
- Un port seul est publié sur un port libre au hasard

environment

L'équivalent de -e : la configuration du service.

```yaml
environment:
  - NODE_ENV=production
  - DB_HOST=db
  - DB_PASSWORD=${DB_PASSWORD}

env_file:
  - .env
```

Les mots de passe vivent dans un fichier .env non versionné, jamais en dur dans compose.yaml.

Les volumes

```yaml
services:
  web:
    image: nginx:alpine
    volumes:
      - "./html:/var/www/html"

  db:
    image: postgres:16-alpine
    volumes:
      - db-data:/var/lib/postgresql/data

volumes:
  db-data:
```

BIND MOUNT

Un chemin de votre disque monté dans le conteneur. Idéal en développement : vos modifications sont visibles immédiatement.

VOLUME NOMMÉ

Géré par Docker et déclaré à la racine du fichier. C'est ce qu'il faut pour les données d'une base : elles survivent à docker compose down.

Rappel : docker compose down -v supprime les volumes nommés, donc les données.

Démarrage et résilience

restart

`"no"`	valeur par défaut : aucun redémarrage
`on-failure`	uniquement si le conteneur sort en erreur
`always`	toujours, y compris au démarrage de la machine
`unless-stopped`	comme always, sauf si vous l'avez arrêté vous-même

depends_on

Fixe l'ordre de démarrage. Attention : il attend que le conteneur démarre, pas que le service soit prêt à répondre.

```
depends_on:
  db:
    condition: service_healthy
```

LE RÉSEAU : CE QUE COMPOSE FAIT SANS QUE VOUS LE DEMANDIEZ

Compose crée un réseau privé pour la stack et y branche tous les services. Chacun est joignable par le nom du service : l'API contacte la base à l'adresse db, sans jamais connaître son adresse IP.

```
environment:
  - DB_HOST=db     # le nom du service
  - REDIS_URL=redis://cache:6379
```

TOUT ASSEMBLER

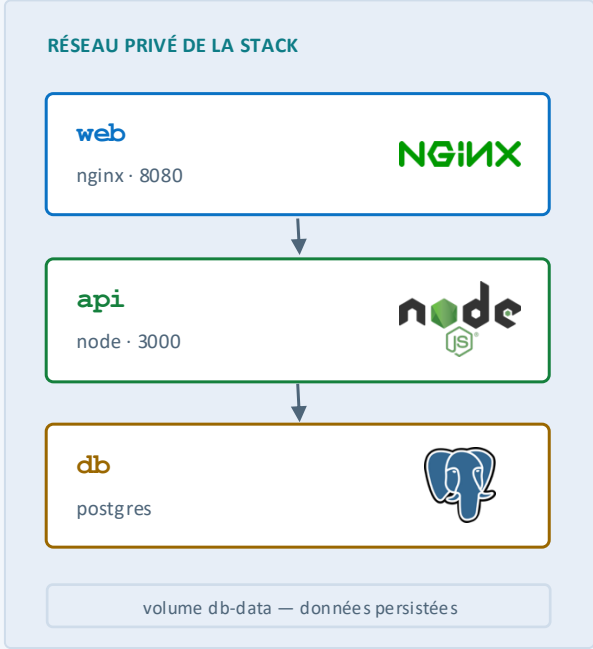
Une stack complète

```yaml
services:
  web:
    image: nginx:alpine
    ports: ["8080:80"]
    volumes: ["./html:/usr/share/nginx/html"]
    depends_on: [api]
    restart: unless-stopped

  api:
    build: ./api
    ports: ["3000:3000"]
    environment:
      - NODE_ENV=production
      - DB_HOST=db
      - DB_PASSWORD=${DB_PASSWORD}
    depends_on: [db]

  db:
    image: postgres:16-alpine
    environment:
      - POSTGRES_PASSWORD=${DB_PASSWORD}
    volumes: [db-data:/var/lib/postgresql/data]

volumes:
  db-data:
```



Un fichier, une commande :
docker compose up -d

TRAVAUX PRATIQUES

TP 6 — Une stack avec Compose

Objectif : orchestrer WordPress et MySQL avec Compose, persister les données, sortir les secrets du fichier.

1	**À la main, d'abord**	Démarrez MySQL puis WordPress avec deux docker run et un réseau commun. Comptez les options à retenir.
2	**La même chose avec Compose**	Décrivez les deux services dans un compose.yaml, démarrez la stack d'une seule commande, lisez les logs de wordpress.
3	**Persister**	Publiez un article, arrêtez la stack, relancez-la. Ajoutez un volume nommé pour MySQL et refaites le test.
4	**Sortir les secrets**	Déplacez le mot de passe dans un fichier .env, référencez-le, vérifiez avec docker compose config.
5	**Ordonner et fiabiliser**	Ajoutez depends_on avec une condition de santé, et restart: unless-stopped sur les deux services.
6	**Nettoyer**	Comparez ce qu'il reste après un down, puis après un down -v. Concluez sur la différence.

Énoncé complet : exercices/Partie 2/Exercice_6.pdf — corrigé : Correction_6.pdf. En cas d'erreur YAML, docker compose config pointe la ligne fautive.

À GARDER SOUS LA MAIN

L'aide-mémoire

Conteneurs

```
docker run -d -p 8080:80 --name web nginx
docker ps -a
docker logs -f web
docker exec -it web sh
docker stop web && docker rm web
```

Images

```
docker images
docker pull postgres:16-alpine
docker build -t mon-api:1.0 .
docker push monlogin/mon-api:1.0
docker rmi mon-api:1.0
```

Compose

```
docker compose up -d
docker compose ps
docker compose logs -f api
docker compose exec api sh
docker compose down    # -v efface les volumes
```

Ménage

```
docker system df
docker container prune
docker image prune -a
docker volume prune
docker system prune -a   # à manier avec soin
```

LA SUITE

Pour aller plus loin

1 Build multi-stage

Compiler dans une première image, ne garder que le binaire dans la seconde. Des images qui passent de 900 Mo à 20 Mo.

2 Healthcheck

Déclarer comment savoir qu'un service est réellement prêt, et fiabiliser depends_on.

3 Docker en CI/CD

Construire et publier l'image automatiquement à chaque commit, puis déployer cette image exacte.

4 Sécurité des images

Analyser ses images (docker scout), ne pas tourner en root, réduire la surface d'attaque.

5 Réseaux avancés

Plusieurs réseaux, isolation d'un service de base de données, reverse proxy en frontal.

6 Orchestration

Quand une machine ne suffit plus : Kubernetes, et ce que Docker vous a déjà appris pour l'aborder.

Documentation de référence : docs.docker.com — claire, à jour, et le premier réflexe à prendre.

Merci

Vos questions sont les bienvenues — et vos conteneurs cassés aussi.

CE QUE VOUS SAVEZ FAIRE

- Lancer et déboguer un conteneur
- Écrire un Dockerfile propre
- Orchestrer une stack avec Compose

Aloïs Marcellin — Lead developer fullstack, Wizards Technologies